

Optimal Stopping in a Finite Reward-Sampling Game

Originally submitted as “Sequential Testing”

AUTHORS

Abhirup Mistry
Aditya Aryan
Deep Jayesh Hariya
Drishti Singla
Mrittika Giri
Nandita Sen

SUPERVISOR

Dr. Arnab Chakraborty

COURSE / UNIT

Statistical Methods II · Indian Statistical Institute, Kolkata

ORIGINAL REPORT DATE

27 March 2024

PUBLIC PORTFOLIO EDITION

September 2026 · Student identifiers removed; substantive report pages otherwise retained from the original submission.

Contents

1	Abstract	1
2	Initial Efforts	1
2.1	Preliminary Idea	1
3	The Optimal Approach	2
3.1	Notations	2
3.2	Backtracking Algorithm	2
3.3	Analysing the Problem	3
3.3.1	Constructing the Optimal strategy	4
3.3.2	Analysing Cutoffs	4
4	Conclusion	5
A	Appendix	6
A.1	Preliminary Idea Algorithm (python)	6
A.2	Backtracking Algorithm (C++)	6
A.3	Cutoff Algorithm (C++)	7
A.4	Expectation of Derived Strategy (python)	7
A.5	Cutoffs	8

1 Abstract

This project delves into the exploration of sequential testing.

The desired test procedure should split the sample space into three parts: accept, reject and continue. This project will explore a simple toy version of this: there are 20 closed boxes, exactly 5 of which contain a Rs 5 coin. We have to pay Re 1 to open a box. We intend to find out the best strategy to maximise the expected gain.

2 Initial Efforts

Our aim is to find a strategy that gives us the maximum expected gain over all the ${}^{20}C_5$ possible combinations of the 5 coin boxes and 15 non-coin boxes.

We are working under the assumption that the coins are uniformly distributed among the boxes, implying that each unopened box has equal probability to contain a coin. Hence, any strategy involving skipping of boxes is equivalent to a strategy where the boxes are opened in order. We therefore omit such cases completely, and only focus on strategies of the latter kind.

2.1 Preliminary Idea

The total number of coins in 20 boxes is 5. Let E_x be the expected number of coins in x boxes and c_x be the no. of coins received in x boxes. Thus, $E_{20} = 5$, $E_{10} = 2.5$. Therefore,

$$E_x = \frac{5}{20} * x = 0.25x$$

Let G_x be the gain after opening x boxes. Note that if all boxes are opened, we get a net gain of +5. So, any strategy which yields a net gain less than 5 over any combination is termed as a 'bad strategy'. Keeping these points in mind, we propose the following strategy:

If $G_x \leq 5$, we open the $(x+1)^{th}$ box.

Else, if $G_x > 5$, and $c_x \leq 0.25x (= E_x)$, we open the $(x+1)^{th}$ box.

Else (i.e, $G_x > 5$, and $c_x > 0.25x$), we stop opening.

Algorithm 1 Initial Strategy

```

1: TotalBoxes  $\leftarrow$  20
2: TotalCoins  $\leftarrow$  5
3: BoxesOpened  $\leftarrow$  0
4: CoinsReceived  $\leftarrow$  0
5: boxes_with_coins  $\leftarrow$  Randomly select TotalCoins unique numbers from 1 to TotalBoxes
6: while BoxesOpened < TotalBoxes do
7:   BoxesOpened  $\leftarrow$  BoxesOpened + 1
8:   if BoxesOpened in boxes_with_coins then
9:     CoinsReceived  $\leftarrow$  CoinsReceived + 1
10:    NetGain  $\leftarrow$  (CoinsReceived * 5) - BoxesOpened
11:    if NetGain  $\leq$  5 then
12:      continue
13:    else if NetGain > 5 and CoinsReceived  $\leq$  0.25 * TotalCoins then
14:      continue
15:    else
16:      break
17:    end if
18:  end if
19: end while
20: print(Total boxes opened:, BoxesOpened)
21: print(Total coins received:, CoinsReceived)
22: print(Net gain:, NetGain)

```

This strategy gives an average net gain of **6.639318885448916** over all ${}^{20}C_5$ possible combinations.

3 The Optimal Approach

Building upon insights gained from our previous unsuccessful attempts, we opted for a fresh perspective to tackle the challenge. Our exploration led us to explore recursive solutions to our optimisation problem.

We examine expected gain at every state of our process and decide whether to stop the process or continue opening. Thus obtaining expected gain at every state of the process, assigning it zero in cases of negative expected gain i.e. deciding not to open any further.

3.1 Notations

Assume total number of boxes to be N , where we have a total of C reward boxes. After opening i ($\leq N$) arbitrary boxes, let x be the the number of reward boxes obtained having monetary value V , and y be the number of empty boxes obtained. Each box can be opened by expenditure of 1 unit currency. Thus we denote a game state as $S(x, y)$.

Let $\sigma_{x,y}$ denote any strategy for the game state $S(x, y)$ and $\Sigma_{x,y}$ denote set of all possible strategies for the game state $S(x, y)$ i.e. $\sigma_{x,y}$ defines a sequence of actions for subsequent game states, and $\Gamma_{\sigma_{x,y}}$ denote the gain of strategy $\sigma_{x,y}$ for game states after $S(x, y)$ on any random permutation of coin boxes.

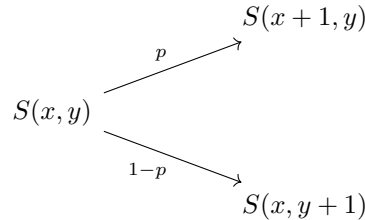
Hence, $\mathbb{E}(\Gamma_{\sigma_{x,y}})$ would be the expected gain of the strategy $\sigma_{x,y}$ over all permutations of coin boxes.

3.2 Backtracking Algorithm

At any game state $S(x, y)$ the game can either proceed further to game state $S(x+1, y)$ or $S(x, y+1)$. Let p be the probability of getting a reward box next i.e. advancing to game state $S(x+1, y)$, hence $1-p$ will be the probability of getting an empty box next i.e. advancing to game state $S(x, y+1)$. p can be calculated as:

$$p = \frac{C-x}{N-x-y} \quad x \leq C, y \leq N-C$$

obtained by dividing the number of remaining reward boxes ($C-x$) by the total number of unopened boxes ($N-x-y$).



Thus for any strategy $\sigma_{x,y}$ at game state $S(x, y)$, it contains information about all possible actions in the subsequent game states, $\sigma_{x+1,y}$ and $\sigma_{x,y+1}$ would contain the tail information about actions after the state $S(x, y)$. Thus it can be defined as sum of indicators as follows:

$$\sigma_{x,y} = \mathbb{I}_{S(x+1,y)} \sigma_{x+1,y} + \mathbb{I}_{S(x,y+1)} \sigma_{x,y+1}$$

Similarly, $\Gamma_{\sigma_{x,y}}$ can be written as following accounting for the expenditure and gains when moving to the respective game states:

$$\Gamma_{\sigma_{x,y}} = \mathbb{I}_{S(x+1,y)} (V-1 + \Gamma_{\sigma_{x+1,y}}) + \mathbb{I}_{S(x,y+1)} (\Gamma_{\sigma_{x,y+1}} - 1)$$

Definition: Now, we define a function $\text{GAIN}(x, y, C, V, N)$

$$\text{GAIN}(x, y, C, V, N) := \begin{cases} \max(\lambda, 0) & x < C \\ 0 & x = C \end{cases}$$

where

$$\lambda = p \cdot (V-1 + \text{GAIN}(x+1, y, C, V, N)) + (1-p) \cdot (-1 + \text{GAIN}(x, y+1, C, V, N))$$

With base case returning 0 if all reward boxes are opened i.e. $x \geq C$

Proposition 1. Function $\text{GAIN}(x, y, C, V, N)$, defined recursively calculates the maximum achievable expected gain at any game state $S(x, y)$ i.e.

$$\text{GAIN}(x, y, C, V, N) = \max_{\sigma_{x,y}} \mathbb{E}(\Gamma_{\sigma_{x,y}})$$

Proof. We prove the proposition by backward induction.

Base case: If we open all coin boxes i.e. $x = C$ then there are no more possible gains. Hence, $\max_{\sigma_{x,y}} \mathbb{E}(\Gamma_{\sigma_{x,y}}) = 0$.

$$x = C \implies \text{GAIN}(x, y, C, V, N) = 0$$

Hence our condition

$$\text{GAIN}(x, y, C, V, N) = \max_{\sigma_{x,y}} \mathbb{E}(\Gamma_{\sigma_{x,y}})$$

is true under base case.

Induction Hypothesis: Assume that at any game state $S(x, y)$ such that $x + y \geq k \ \forall k \leq N$

$$\text{GAIN}(x, y, C, V, N) = \max_{\sigma_{x,y}} \mathbb{E}(\Gamma_{\sigma_{x,y}})$$

Inductive Step: To prove: $\text{GAIN}(x, y, C, V, N) = \max_{\sigma_{x,y}} \mathbb{E}(\Gamma_{\sigma_{x,y}})$ such that $x + y = k - 1$

Since we take any strategy $\sigma_{x,y}$, to be such that we never skip any of the boxes, therefore we always move to the strategies $\sigma_{x+1,y}$ and $\sigma_{x,y+1}$ with probability p and $1 - p$ respectively, as defined above.

$$\mathbb{E}[\Gamma_{\sigma_{x,y}}] = \mathbb{E}[\mathbb{I}_{S(x+1,y)} (V - 1 + \Gamma_{\sigma_{x+1,y}}) + \mathbb{I}_{S(x,y+1)} (\Gamma_{\sigma_{x,y+1}} - 1)]$$

Now since $\mathbb{I}_{S(x+1,y)}$ depends on next box, but $\Gamma_{\sigma_{x+1,y}}$ calculates gain of $\sigma_{x+1,y}$ from states after $S(x + 1, y)$. Hence $\mathbb{I}_{S(x+1,y)}$ and $\Gamma_{\sigma_{x+1,y}}$ are independent, similarly $\mathbb{I}_{S(x,y+1)}$ and $\Gamma_{\sigma_{x,y+1}}$ are independent.

$$\begin{aligned} &= \mathbb{E}[\mathbb{I}_{S(x+1,y)}] \mathbb{E}[(V - 1 + \Gamma_{\sigma_{x+1,y}})] + \mathbb{E}[\mathbb{I}_{S(x,y+1)}] \mathbb{E}[(\Gamma_{\sigma_{x,y+1}} - 1)] \\ &= p (V - 1 + \mathbb{E}[\Gamma_{\sigma_{x+1,y}}]) + (1 - p) (\mathbb{E}[\Gamma_{\sigma_{x,y+1}}] - 1) \end{aligned}$$

Therefore, for maximum expectation over all $\sigma_{x,y}$ we have

$$\begin{aligned} \max(\mathbb{E}[\Gamma_{\sigma_{x,y}}]) &= \max(p (V + \mathbb{E}[\Gamma_{\sigma_{x+1,y}}] - 1) + (1 - p) (\mathbb{E}[\Gamma_{\sigma_{x,y+1}}] - 1)) \\ &= p (V + \max(\mathbb{E}[\Gamma_{\sigma_{x+1,y}}]) - 1) + (1 - p) (\max(\mathbb{E}[\Gamma_{\sigma_{x,y+1}}]) - 1) \\ &= p (V + \text{GAIN}(x + 1, y, C, V, N) - 1) + (1 - p) (\text{GAIN}(x, y + 1, C, V, N) - 1) \\ &= \text{GAIN}(x, y, C, V, N) \end{aligned}$$

This proves that the gain function gives the maximum return for $x + y = k - 1$ as well. Thus, by backward induction, we have proved that the function $\text{GAIN}(x, y, C, V, N)$, defined recursively, calculates the maximum achievable expected gain at any game state $S(x, y)$. □

Algorithm 2 Recursion Algorithm

```

1: function GAIN( $x, y, C, V, N$ )
2:   if  $x \geq C$  or  $y > N - C$  then
3:     return 0
4:   end if
5:    $p \leftarrow \frac{C-x}{N-x-y}$ 
6:    $\lambda \leftarrow p \times (V - 1 + \text{GAIN}(x + 1, y, C, V, N)) + (1 - p) \times (\text{GAIN}(x, y + 1, C, V, N) - 1)$ 
7:   return  $\max(\lambda, 0)$ 
8: end function

```

3.3 Analysing the Problem

The problem deals with a specific case of our generalised version of the problem setup. For the specific case of $C = 5$, $V = ₹5$, and $N = 20$, the gain function is called, with initial parameters:

$x = 0$ (No reward boxes obtained)

$y = 0$ (No empty boxes obtained)

The given backtracking algorithm yields the maximum achievable expected gain for the game state $S(0, 0)$ i.e.

$\text{GAIN}(0, 0, 5, 5, 20) = \mathbf{7.6875644994840036}$

3.3.1 Constructing the Optimal strategy

To construct the optimal strategy, we establish cutoffs by determining the maximum number of boxes that can be opened while ensuring a positive expected gain. We employed our backtracking strategy to compute a cutoff value, utilizing the function $\text{GAIN}(x, y, C, V, N)$.

The process involves iterating over the total number of boxes, denoted as n , for a fixed number of reward boxes, denoted as c . At each iteration, we calculate the gain and return the maximal positive n for a positive gain. This calculation is performed for varying values of c , where $c \in \{1, 2, 3, 4, 5\}$.

Algorithm 3 Cutoff Algorithm

```

1:  $n \leftarrow 1$ 
2: for  $c = 1, \dots, 5$  do
3:    $G \leftarrow 1$ 
4:   while  $G > 0$  do
5:      $n \leftarrow n + 1$ 
6:      $G \leftarrow \text{GAIN}(0, 0, c, 5, n)$ 
7:   end while
8:   return  $[c, n - 1]$ 
9: end for

```

Calculating cutoffs from the above algorithm we get:

Reward Boxes(c)	1	2	3	4	5
Cutoff(n_c)	8	15	21	27	33

Now we construct our strategy for the problem statement using these cutoff values. Since the cutoff for three reward boxes exceeds the total number of boxes, we aim to open boxes until we obtain three reward boxes. Subsequently, we initiate another opening spree if the number of remaining boxes is less than or equal to the cutoff for two boxes, denoted as n_2 (which is 15 in this case), with the objective of obtaining one more reward. Finally, if the remaining number of boxes is less than or equal to the cutoff for one box, denoted as n_1 (which is 8) after the second opening spree, we continue opening boxes until completion. This sequential approach maximizes our expected gain while efficiently utilizing the determined cutoff values.

Algorithm 4 Derived strategy

```

1:  $\alpha \leftarrow 5$ 
2: for  $i = 1, \dots, 20$  do
3:   Open the box
4:   if A coin is found then
5:      $\alpha = \alpha - 1$ 
6:   end if
7:   if  $n_\alpha < 20 - i$  then
8:     break
9:   end if
10: end for

```

After applying this strategy we saw that the expected gain of this strategy coincides with the value of $\text{GAIN}(0, 0, 5, 5, 20)$ (refer to A.4). Hence we conclude that this is our optimal strategy which maximises expected gain over all possible permutations of coin boxes.

3.3.2 Analysing Cutoffs

We investigated the convergence of cutoff values with respect to the number of reward boxes. Employing our algorithm, cutoff values were computed for varying quantities of reward boxes. Subsequently, we generated a scatter plot to visually represent the relationship between the number of reward boxes and their corresponding cutoff values:

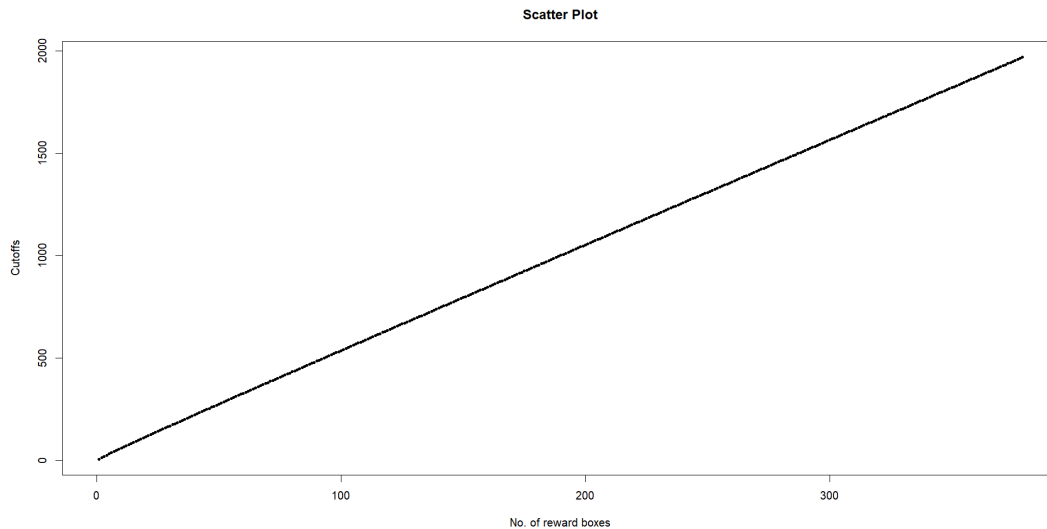


Figure 3.1: Scatter plot

Our analysis reveals a consistent linear increase in cutoff values as the number of reward boxes escalates. The observed linear trend in cutoff values indicates a lack of convergence in the sequence concerning higher numbers of reward boxes.

4 Conclusion

In conclusion, our project marked a significant success as we found an optimal strategy for opening boxes. This accomplishment showcases the potential of our approach in maximising the expected gain over all possible permutations of the arrangement of the boxes.

While our current approach excelled in finding the maximum optimal solution possible, the backtracking algorithm is still computationally heavy, as it iteratively calculates solution at each point. Although we used memoization to store values which exponentially decreased the calculations at each recursion, we were still unable to compute solutions for higher values. Furthermore, our current project has been based on the assumption of a uniform distribution of reward boxes. However, we have not yet explored the potential implications of varying the distribution of reward boxes to follow different probability distributions. This exploration could unveil valuable insights into how the distribution pattern affects the optimal strategy and expected gain in our problem scenario.

In this project, we analysed different methods to maximise our expected gain, exploring different intuitions. Moreover, the implementation of backtracking algorithm has introduced us to a new domain of optimisation techniques.

A Appendix

A.1 Preliminary Idea Algorithm (python)

```
import random

# Initialize variables
TotalBoxes = 20
TotalCoins = 5
BoxesOpened = 0
CoinsReceived = 0

# Place coins randomly in the boxes
boxes_with_coins = random.sample(range(1, TotalBoxes + 1), TotalCoins)

while BoxesOpened < TotalBoxes:
    # Open a box
    BoxesOpened += 1
    # Check if the opened box contains a coin
    if BoxesOpened in boxes_with_coins:
        CoinsReceived += 1
        # Calculate NetGain
        NetGain = (CoinsReceived * 5) - BoxesOpened
        # Conditions to continue opening boxes or break from the loop
        if NetGain <= 5:
            continue
        elif NetGain > 5 and CoinsReceived <= 0.25 * TotalCoins:
            continue
        else:
            break
    # Print results
print("Total boxes opened:", BoxesOpened)
print("Total coins received:", CoinsReceived)
print("Net gain:", NetGain)
```

A.2 Backtracking Algorithm (C++)

```
#include <bits/stdc++.h>
using namespace std;

map<vector<int>, double> memMap;
double gain(int x, int y, int C, int V, int N) {
    vector<int> key {x,y,C,N};
    if (memMap.find(key) != memMap.end())
        return memMap[key];

    if (x >= C || y > N - C)
        return memMap[key] = 0;

    double p = static_cast<double>(C - x) / (N - x - y);
    double g = p * (V - 1 + gain(x + 1, y, C, V, N))
        + (1 - p) * (gain(x, y + 1, C, V, N) - 1);
    return memMap[key] = max(g, 0.0);
}

int main(){
    cout << gain(0,0,5,5,20);
}
```


A.3 Cutoff Algorithm (C++)

```
#include <bits/stdc++.h>
#include "gain.h"
using namespace std;

int main() {
    int k=1;
    for (int c = 1; c <= 1000; ++c) {
        int n = k;
        double G = 4 * n;
        while (G > 0) {
            ++n;
            G = gain(0, 0, c, 5, n);
        }
        k = n;
        cout << "[" << c << "," << n - 1 << "]" << endl;
    }
}
```

A.4 Expectation of Derived Strategy (python)

```
L=[]
for a in range(1,17):
    for b in range(a+1, 18):
        for c in range(b+1, 19):
            for d in range(c+1, 20):
                for e in range(d+1,21):
                    L.append([a,b,c,d,e])

P = []

def profit(l):
    p = 15 - l[2]
    if (20 - l[2]) > 15:
        return p
    p += (5 - l[3] + l[2])
    if (20 - l[3])>8:
        return p
    p += (5-l[4]+l[3])
    return p
for i in L:
    P.append(profit(i))
print(sum(P)/len(P))
```

A.5 Cutoffs

i	n_i	i	n_i	i	n_i	i	n_i	i	n_i	i	n_i	i	n_i	i	n_i
1	8	36	202	71	387	106	569	141	750	176	930	211	1110	246	1289
2	15	37	208	72	392	107	574	142	755	177	935	212	1115	247	1294
3	21	38	213	73	397	108	579	143	760	178	940	213	1120	248	1299
4	27	39	218	74	402	109	584	144	765	179	945	214	1125	249	1304
5	33	40	224	75	407	110	589	145	770	180	950	215	1130	250	1309
6	39	41	229	76	413	111	595	146	775	181	955	216	1135	251	1314
7	45	42	234	77	418	112	600	147	780	182	961	217	1140	252	1320
8	50	43	240	78	423	113	605	148	786	183	966	218	1145	253	1325
9	56	44	245	79	428	114	610	149	791	184	971	219	1151	254	1330
10	62	45	250	80	434	115	615	150	796	185	976	220	1156	255	1335
11	67	46	255	81	439	116	620	151	801	186	981	221	1161	256	1340
12	73	47	261	82	444	117	626	152	806	187	986	222	1166	257	1345
13	78	48	266	83	449	118	631	153	811	188	991	223	1171	258	1350
14	84	49	271	84	454	119	636	154	817	189	997	224	1176	259	1355
15	89	50	276	85	460	120	641	155	822	190	1002	225	1181	260	1361
16	95	51	282	86	465	121	646	156	827	191	1007	226	1186	261	1366
17	100	52	287	87	470	122	651	157	832	192	1012	227	1192	262	1371
18	106	53	292	88	475	123	657	158	837	193	1017	228	1197	263	1376
19	111	54	298	89	480	124	662	159	842	194	1022	229	1202	264	1381
20	117	55	303	90	486	125	667	160	847	195	1027	230	1207	265	1386
21	122	56	308	91	491	126	672	161	853	196	1033	231	1212	266	1391
22	128	57	313	92	496	127	677	162	858	197	1038	232	1217	267	1396
23	133	58	319	93	501	128	682	163	863	198	1043	233	1222	268	1401
24	138	59	324	94	506	129	688	164	868	199	1048	234	1227	269	1407
25	144	60	329	95	512	130	693	165	873	200	1053	235	1233	270	1412
26	149	61	334	96	517	131	698	166	878	201	1058	236	1238	271	1417
27	154	62	339	97	522	132	703	167	883	202	1063	237	1243	272	1422
28	160	63	345	98	527	133	708	168	889	203	1068	238	1248	273	1427
29	165	64	350	99	532	134	713	169	894	204	1074	239	1253	274	1432
30	170	65	355	100	537	135	719	170	899	205	1079	240	1258	275	1437
31	176	66	360	101	543	136	724	171	904	206	1084	241	1263	276	1442
32	181	67	366	102	548	137	729	172	909	207	1089	242	1268	277	1447
33	186	68	371	103	553	138	734	173	914	208	1094	243	1274	278	1453
34	192	69	376	104	558	139	739	174	919	209	1099	244	1279	279	1458
35	197	70	381	105	563	140	744	175	925	210	1104	245	1284	280	1463

i	n_i	i	n_i	i	n_i
281	1468	314	1637	347	1805
282	1473	315	1642	348	1810
283	1478	316	1647	349	1815
284	1483	317	1652	350	1820
285	1488	318	1657	351	1825
286	1493	319	1662	352	1830
287	1499	320	1667	353	1836
288	1504	321	1672	354	1841
289	1509	322	1677	355	1846
290	1514	323	1682	356	1851
291	1519	324	1688	357	1856
292	1524	325	1693	358	1861
293	1529	326	1698	359	1866
294	1534	327	1703	360	1871
295	1539	328	1708	361	1876
296	1545	329	1713	362	1881
297	1550	330	1718	363	1887
298	1555	331	1723	364	1892
299	1560	332	1728	365	1897
300	1565	333	1734	366	1902
301	1570	334	1739	367	1907
302	1575	335	1744	368	1912
303	1580	336	1749	369	1917
304	1585	337	1754	370	1922
305	1591	338	1759	371	1927
306	1596	339	1764	372	1932
307	1601	340	1769	373	1938
308	1606	341	1774	374	1943
309	1611	342	1779	375	1948
310	1616	343	1785	376	1953
311	1621	344	1790	377	1958
312	1626	345	1795	378	1963
313	1631	346	1800	379	1968